

UNIVERSIDADE DO SUL DE SANTA CATARINA - UNISUL

Curso de Ciencias da Computacao

Disciplina: Computacao Grafica



SEA SHARK ADVENTURE

Documentacao Final - A3

Jogo Plataforma 2D com Tematica Submarina

Integrante:

Henrique Antony Pacheco da Silva - RA: 1072323727

Biguaçu, SC - Junho de 2026

Acesse o jogo: <https://sea-adventure.vercel.app/>

1. INTRODUÇÃO E OBJETIVOS

O projeto Sea Shark Adventure foi desenvolvido como trabalho final da disciplina de Computação Gráfica, do curso de Ciências da Computação da UNISUL. Trata-se de um jogo plataforma 2D de rolagem lateral (side-scrolling) com ambientação submarina, executado diretamente no navegador web sem necessidade de instalação ou plugins externos.

O personagem principal é um híbrido entre humano e tubarão - um guerreiro que navega pelas profundezas do oceano enfrentando criaturas marinhas, coletando itens e avançando por uma fase com dificuldade progressiva. O jogo é acessível publicamente em <https://sea-adventure.vercel.app/>.

1.1 OBJETIVOS GERAIS

Aplicar na prática os conceitos estudados na disciplina de Computação Gráfica, abrangendo pipeline gráfico, sprites, animações por sprite sheet, renderização em tempo real e física 2D, por meio do desenvolvimento completo de um jogo funcional rodando no navegador.

1.2 OBJETIVOS ESPECÍFICOS

- Implementar um pipeline gráfico completo utilizando HTML5 Canvas API e JavaScript puro;
- Desenvolver animações de sprite sheet para o personagem principal com múltiplos estados (idle, run, jump, attack, crouch, hurt);
- Criar inimigos desenhados via Canvas com comportamentos distintos de IA baseados em máquinas de estado;
- Implementar sistema de projéteis (lanças de tridente) para o inimigo Tritão;
- Desenvolver parallax scrolling com 5 camadas para profundidade visual do cenário submarino;
- Implementar sistema de save/load via localStorage para persistência de progresso;
- Publicar o jogo em plataforma web acessível (Vercel) para avaliação e testes online.

2. FUNDAMENTAÇÃO TEÓRICA

2.1 JOGOS PLATAFORMA 2D E SIDE-SCROLLING

Jogos de plataforma 2D constituem um dos gêneros mais influentes da história dos videogames. Caracterizam-se pela movimentação lateral do personagem sobre plataformas, com câmera que acompanha o jogador horizontalmente. O modelo side-scrolling, popularizado por títulos clássicos, permanece altamente relevante no mercado independente contemporâneo e é amplamente utilizado como projeto de aprendizado em desenvolvimento de jogos (NOVAK, 2010).

2.2 PIPELINE GRÁFICO E CANVAS API

O pipeline grafico e o processo pelo qual dados de estado do jogo sao transformados em pixels na tela a cada frame. Na HTML5 Canvas API, o contexto 2D oferece metodos para desenhar formas geometricas, imagens e sprites diretamente no bitmap. O ciclo game loop - limpar o canvas, atualizar estado, redesenhar - e executado com `setInterval` a 60 quadros por segundo, garantindo animacao fluida. No Sea Shark Adventure, toda a renderizacao ocorre em um unico canvas de 816x480 pixels.

2.3 SPRITES E ANIMACAO POR SPRITE SHEET

Sprites sao imagens utilizadas para representar personagens e objetos. O personagem principal utiliza sprite sheets externos (arquivos PNG separados por animacao: idle, run, jump, attack, crouch, hurt, em versoes esquerda e direita), com cada frame medindo 100x59 pixels. A animacao e obtida avancando o frame atual a cada 100ms, recortando a regioa correspondente do sprite sheet com `drawImage` e os parametros de posicao e dimensao corretos (SCHUYTEMA, 2008).

2.4 FISICA DE JOGO E COLISAO AABB

A simulacao de fisica controla posicao, velocidade e aceleracao das entidades. O sistema implementa gravidade constante (0.5 unidades/frame), velocidade de pulo (-10) e um modo especial de hover jump desbloqueavel. A deteccao de colisao utiliza o metodo AABB (Axis-Aligned Bounding Box), verificando sobreposicao entre retangulos. O personagem possui uma hitbox reduzida (offset de 25px, largura de 50px) menor que seu sprite, tornando o combate mais preciso e justo ao jogador.

2.5 PARALLAX SCROLLING

O parallax e uma tecnica visual em que camadas de fundo se movem em velocidades diferentes proporcional ao `scrollX` do nivel, criando ilusao de profundidade tridimensional em cenarios 2D. No Sea Shark Adventure, cinco camadas do `parallax_mountain_pack` sao renderizadas com fator de velocidade exponencial ($0.005 * scrollX * 2^i$), onde as camadas mais distantes se movem mais lentamente. O tile-wrapping garante que as imagens se repitam continuamente ao longo do nivel.

2.6 INTELIGENCIA ARTIFICIAL DE INIMIGOS

A IA dos inimigos e implementada como maquinas de estado finito (FSM), alternando entre estados de patrulha, hurt e dead. Em patrulha, os inimigos se movem dentro de um intervalo definido (`patrolLeft/patrolRight`), detectam bordas de plataforma, reverterem direcao ao bater em paredes e aplicam gravidade. O Tritao possui logica adicional de disparo de projeteis baseada na distancia ao jogador. A colisao entre inimigos e o mundo usa o mesmo sistema AABB do personagem principal.

3. METODOLOGIA

O desenvolvimento do Sea Shark Adventure seguiu uma abordagem iterativa e incremental, com ciclos curtos de implementaçao, teste e refinamento. Por tratar-se de um projeto individual, as decisoes de design e arquitetura foram tomadas diretamente no código, priorizando a entrega de um produto funcional e jogável.

3.1 FASES DE DESENVOLVIMENTO

- Fase 1 - Prototipagem: implementação da estrutura básica com canvas, game loop via setInterval, movimentação WASD, gravidade e colisão com tiles;
- Fase 2 - Criação dos inimigos: desenvolvimento dos três tipos de inimigos (Caranguejo, Tritão, Polvo) com comportamentos de IA distintos, barra de HP e sistema de dano;
- Fase 3 - Sprites e animações: integração dos sprite sheets do personagem (12 animações), sprite sheet do Polvo com animações de walk/hurt/death, e sprites procedurais dos demais inimigos via Canvas;
- Fase 4 - Sistemas de jogo: implementação do HUD (vidas, pontuação, kills), tela de título com menu navegável, menu de pausa, tela de game over e tela de conclusão de fase;
- Fase 5 - Conteúdo e balanceamento: criação do mapa de tiles (15 linhas x 108 colunas), posicionamento de 14 inimigos, item de hover jump e item Golden Cross, sistema de projéteis do Tritão;
- Fase 6 - Publicação: deploy na Vercel com acesso público em <https://sea-adventure.vercel.app/>.

3.2 TECNOLOGIAS E FERRAMENTAS UTILIZADAS

Tecnologia	Finalidade
HTML5 / Canvas API	Estrutura da página e renderização de todos os elementos gráficos
JavaScript (ES6+)	Lógica do jogo, física, IA, game loop e gerenciamento de estados
CSS3	Estilização básica da página host (style.css)
Web Audio API	Reprodução da trilha sonora (MusicGame.mp3, moonlight.mp3)
localStorage	Persistência de save: pontuação, kills e progresso de fase
Vercel	Hospedagem e deploy contínuo do projeto na web
Sprite sheets PNG	Animações do personagem principal (12 estados) e do Polvo

4. DESCRIÇÃO DO SISTEMA

4.1 ESTRUTURA DO PROJETO

O projeto é composto por um arquivo HTML de entrada (index.html), uma folha de estilos (css/style.css), o arquivo principal de lógica (js/game.js com 1726 linhas) e uma pasta de assets contendo os sprite sheets do personagem, sprites do Polvo, imagens de parallax, tileset, fontes e arquivos de áudio. Toda a lógica do jogo está concentrada no game.js, organizada em seções comentadas: Constants, Level Data, Physics, Enemy, HUD, Levels/Enemies e a classe principal GameEngine.

4.2 CLASSE GAMEENGINE E GAME LOOP

O núcleo do jogo é a classe GameEngine, instanciada com o canvas, as imagens precarregadas e o elemento de áudio. O game loop é executado via setInterval a 60

FPS (16.6ms/frame). Em cada tick, o sistema verifica o estado atual (title, playing, paused, gameover, levelcomplete) e executa a rotina correspondente. No estado playing, a ordem é: renderizar parallax, renderizar tiles, renderizar e atualizar inimigos, atualizar projeteis, processar input, aplicar fisica, renderizar hero, verificar itens, processar combate, renderizar HUD e efeitos.

4.3 SISTEMA DE MOVIMENTACAO E FISICA

O personagem se move com velocidade de 240 unidades/segundo (teclas A/D). A fisica aplica gravidade de 0.5/frame quando o personagem não está no chão. O pulo usa velocidade inicial de -10. Há um modo especial de hover jump (desbloqueado pelo item de asa) que permite pulo contínuo enquanto a tecla W é mantida, com velocidade de -4. O knockback ao receber dano empurra o personagem com velocidade 8, decaindo a 80% por frame até zerar. A hitbox do personagem (25px de offset, 50px de largura) é menor que o sprite (100px) para fairness de colisao.

4.4 INIMIGOS E INTELIGENCIA ARTIFICIAL

Foram desenvolvidos três tipos de inimigos, cada um com atributos e comportamentos distintos:

Inimigo (nome interno)	Atributos e Comportamento
Caranguejo (slime)	HP: 40 Velocidade: 65 Pontos: 60. Patrulha horizontal simples, detecta bordas de plataforma e paredes. Desenhado via Canvas API com pernas animadas por seno, garra e olhos em palito.
Tritao (goblin)	HP: 80 Velocidade: 110 Pontos: 180. Patrulha rapida. Dispara projéteis de tridente quando o jogador está entre 60 e 320px de distancia, com cooldown de 2400-3200ms. Desenhado via Canvas com elmo/coroa, escamas e tridente.
Polvo (knight)	HP: 180 Velocidade: 55 Pontos: 450. O mais resistente. Usa sprite sheet externo com animacoes de walk, hurt e death. Flip horizontal para direcao. Barra de HP visivelmente maior.

Todos os inimigos compartilham o sistema FSM (patrol -> hurt -> dead), aplicam gravidade, detectam colisao com tiles e possuem barra de HP dinamica (verde/amarela/vermelha). O kill por pisada (stomp) mata o inimigo instantaneamente e aplica pulo de rebote ao jogador.

4.5 SISTEMA DE PROJETEIS

O Tritao dispara lancas de tridente representadas por objetos com posicao, velocidade horizontal (280 unidades/s na direcao do inimigo), timer e lifetime (2200ms). Os projeteis são desenhados com rotacao animada via ctx.rotate, simulando o tridente girando em voo. A colisao com o jogador é verificada por AABB a cada frame. Projeteis expirados ou que acertaram o jogador são removidos do array.

4.6 SISTEMA DE COMBATE DO JOGADOR

O ataque (tecla K) ativa as animacoes heroAttackLeft/Right. A hitbox de ataque é um retangulo de 70px de largura posicionado a frente do personagem. O sistema usa um Set (attackHitSet) para garantir que cada inimigo receba dano apenas uma vez por animacao de ataque, evitando hits multiplos no mesmo swing. Cada acerto aplica 25 de dano e exibe um numero flutuante de dano em amarelo sobre o inimigo atingido.

4.7 CENARIO, TILES E PARALLAX

O nível é definido como uma matriz de 15 linhas por 108 colunas de tiles 32x32px, totalizando 3456x480 pixels de nível. Diferentes valores de tile (1-12) referenciam posições no Tileset.png externo. Valores especiais incluem: tile 10 (item de asa/hoverjump), tile 12 (Golden Cross, vale 10.000 pontos) e tiles 2-9 (decorações de plataforma). O parallax usa 5 camadas do parallax_mountain_pack, renderizadas com velocidade exponencial por camada.

4.8 HUD E INTERFACE

O HUD exibe: contador de vidas com corações (até 3), indicador de hover jump quando ativo, pontuação com 6 dígitos formatados e contador de inimigos derrotados. O overlay de dano (vermelho semitransparente piscante) indica que o jogador foi atingido. O jogo possui quatro telas de estado: título (com menu de Novo Jogo / Continuar), playing, pausa (com opções Continuar/Resetar Fase) e game over. O progresso é salvo automaticamente ao completar a fase via localStorage.

5. RESULTADOS E LIMITAÇÕES

5.1 RESULTADOS OBTIDOS

O projeto Sea Shark Adventure foi concluído com sucesso, resultando em um jogo funcional acessível em <https://sea-adventure.vercel.app/>. Os principais resultados alcançados foram:

- Pipeline gráfico completo implementado com Canvas API: parallax, tiles, sprites animados, HUD, efeitos visuais e números de dano flutuantes;
- Personagem com 12 animações distintas controladas por sprite sheets externos, com lógica de flip e ciclo de frames por estado;
- Três tipos de inimigos com comportamentos de IA distintos via FSM, incluindo sistema de projéteis para o Tritão;
- Colisão AABB funcional para personagem, inimigos, projéteis e tiles do cenário;
- Sistema de save/load via localStorage, preservando pontuação e kills entre sessões;
- Menus completos (título, pausa, game over, level complete) com navegação por teclado;
- Nível completo com 14 inimigos posicionados, itens coletáveis e progressão até a tela de conclusão;
- Deploy bem-sucedido na Vercel com acesso público e sem necessidade de instalação.

5.2 LIMITAÇÕES IDENTIFICADAS

Apesar dos resultados positivos, algumas limitações foram identificadas:

- Código monolítico: toda a lógica está em um único arquivo game.js (1726 linhas). A modularização em arquivos separados (player.js, enemies.js, render.js etc.) está planejada como trabalho futuro;

- Suporte mobile ausente: o jogo requer teclado fisico; nao ha controles touch nem adaptacao para viewport de dispositivos moveis;
- Fase unica: apenas um nivel foi desenvolvido dentro do escopo do projeto. A adicao de novos biomas e fases esta reservada para iteracoes futuras;
- Deteccao de colisao simplificada: o metodo AABB pode apresentar casos de 'corner clipping' em colisoes diagonais entre o personagem e as bordas dos tiles;
- IA sem pathfinding: os inimigos utilizam patrulha linear simples sem algoritmos de navegacao, podendo ficar presos em geometrias complexas;
- Polvo sem ataque ativo: o inimigo knight (Polvo) atualmente patrulha e causa dano por contato, sem animacao de ataque especial implementada.

6. POSSIVEIS TRABALHOS FUTUROS

O Sea Shark Adventure estabelece uma base solida para expansoes futuras. As principais direcoes planejadas para continuidade do projeto sao:

6.1 REFATORACAO E ARQUITETURA

- Modularizacao do codigo em arquivos JS separados: player.js, enemies.js, collision.js, render.js, audio.js e levels.js, facilitando manutencao e adicao de conteudo;
- Uso de ES6 modules (import/export) para dependencias explicitas entre modulos;
- Separacao dos dados de nivel (LEVEL_DATA) em arquivos JSON externos, permitindo criacao de fases sem alterar o codigo principal.

6.2 VERSAO MOBILE

- Implementacao de controles touch com botoes virtuais sobrepostos ao canvas;
- Adaptacao do viewport e escala do canvas para telas de smartphones (responsive canvas);
- Suporte a API de Gamepad do navegador para controles fisicos bluetooth.

6.3 EXPANSAO DE CONTEUDO

- Novas fases: zona abissal com bioluminescencia, interior de naufragio e recife de coral;
- Novos inimigos: peixe-baiacu com mecanica de inflacao em area, medusa eletrica com padrao de ataque em pulso e tubarao branco como mini-boss;
- Sistema de boss com multiplas fases de comportamento e ataques especiais;
- Mais itens coletaveis: escudo de bolha, pocao de vida e velocidade aumentada.

6.4 MELHORIAS TECNICAS

- Migracao do game loop de setInterval para requestAnimationFrame para sincronizacao com o refresh rate do monitor;
- Implementacao de pathfinding (A*) para IA dos inimigos navegarem em geometrias complexas;

- Sistema de particulas para bolhas, impactos de ataque e trilha do personagem nadando;
- Ranking online com persistencia em servidor para comparacao de pontuacoes entre jogadores.

7. REFERENCIAS BIBLIOGRAFICAS

NOVAK, Jeannie. Desenvolvimento de Games. Sao Paulo: Cengage Learning, 2010.

SCHUYTEMA, Paul. Design de Games: Uma abordagem pratica. Sao Paulo: Cengage Learning, 2008.

SILVA, Mauricio Samy. JavaScript: Guia do Programador. Sao Paulo: Novatec, 2019.

FLANAGAN, David. JavaScript: O Guia Definitivo. Porto Alegre: Bookman, 2021.

MDN WEB DOCS. Canvas API. Disponivel em: https://developer.mozilla.org/en-US/docs/Web/API/Canvas_API. Acesso em: junho 2026.

MDN WEB DOCS. Web Audio API. Disponivel em: https://developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API. Acesso em: junho 2026.

Sea Shark Adventure - Jogo publicado. Disponivel em: <https://sea-adventure.vercel.app/>. Acesso em: junho 2026.